

CLASE: CONOCIENDO NODE Y EXPRESS CON ES MODULES

1. INICIO

Analicemos

Una organización necesita crear un servidor que responda distintas rutas: inicio, productos, contacto y una futura API. Con Node puro se puede, pero el código se vuelve poco mantenible. Usaremos Express para resolverlo mejor.

2. BLOQUE 1: JAVASCRIPT EN EL SERVIDOR

En frontend, JavaScript responde a eventos del usuario: clics, formularios, cambios en el DOM. En backend, JavaScript responde a solicitudes HTTP: una ruta, una consulta, una petición de datos.

Frontend:

Usuario hace clic -> JavaScript cambia la página

Backend:

Usuario entra a una URL -> Servidor responde información

Mini ejemplo

Crear archivo app.js:

```
console.log("JavaScript ejecutándose desde Node.js");
```

Ejecutar:

node app.js

¿Dónde aparece el resultado?

En la terminal.

3. BLOQUE 2: CONFIGURAR PROYECTO MODERNO CON ES MODULES

Como desde ahora usaremos import y no require, esta parte es clave.

En muchos ejemplos antiguos de Node verán require. Nosotros trabajaremos con sintaxis moderna de JavaScript: import y export. Para eso debemos indicarle a Node que nuestro proyecto usará módulos ES.

Paso 1: crear proyecto

```
mkdir clase-node-express  
cd clase-node-express  
npm init -y
```

Paso 2: modificar package.json

Agregar:

```
{  
  "type": "module"  
}
```

Un package.json inicial podría quedar así:

```
{  
  "name": "clase-node-01",  
  "version": "1.0.0",  
  "main": "app.js",  
  "scripts": {  
    "test": "echo \\\"Error: no test specified\\\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "type": "module",  
  "description": ""  
}
```

Explicación breve

```
"type": "module"
```

le dice a Node:

Este proyecto usará import y export.

4. BLOQUE 3: MOTOR V8 Y ENTORNO NODE

V8 ejecuta JavaScript. Chrome lo usa dentro del navegador. Node lo usa fuera del navegador y le suma herramientas propias de servidor.

JavaScript es el lenguaje.
V8 ejecuta JavaScript.
Node usa V8 para correr JavaScript en el servidor.

Pregunta rápida

¿Node.js es un lenguaje nuevo?

Respuesta:

No. El lenguaje sigue siendo JavaScript.
Node es el entorno donde se ejecuta.

5. BLOQUE 4: ASINCRONÍA, CALLBACKS Y EVENT LOOP

Node trabaja muy bien con tareas que pueden demorar: leer archivos, consultar una base de datos o consumir una API. En lugar de quedarse detenido esperando, delega la tarea y continúa.

Ejemplo práctico 1: asincronía básica

Crear asincronia.js:

```
console.log("1. Inicio del programa");

setTimeout(() => {
  console.log("2. Tarea asíncrona finalizada");
}, 2000);

console.log("3. Fin del programa");
```

Ejecutar:

node asincronia.js

Resultado esperado

1. Inicio del programa
3. Fin del programa
2. Tarea asíncrona finalizada

Esto demuestra que Node no espera bloqueado. Continúa ejecutando el programa y luego vuelve a la tarea pendiente.

6. BLOQUE 5: FILE SYSTEM CON ES MODULES

Node puede leer y escribir archivos. Esto será importante porque antes de conectarnos a bases de datos, podemos practicar persistencia simple usando archivos .txt o .json.”

Ejercicio práctico 2: crear archivo con fs

Crear crearArchivo.js:

```
import fs from 'fs';

const contenido = `Producto: Notebook
Precio: 750000
Stock: 10`;

fs.writeFile('./producto.txt', contenido, (error) => {
  if (error) {
    console.log("No se pudo crear el archivo");
    console.log(error.message);
    return;
  }

  console.log("Archivo creado correctamente");
});

console.log("Última línea del programa");
```

Ejecutar:

```
node crearArchivo.js
```

Resultado esperado

Se crea:

```
producto.txt
```

Con contenido:

```
Producto: Notebook
Precio: 750000
Stock: 10
```

¿Por qué aparece primero “Última línea del programa” y después “Archivo creado correctamente”?

Porque writeFile es asíncrono y ejecuta el callback cuando termina.

7. BLOQUE 6: LECTURA DE ARCHIVO CON CALLBACK

Crear archivo mensaje.txt

Hola desde un archivo leído con Node.js

Crear leerArchivo.js

```
"use strict";

// importamos el modulo http usando ES modules
// http viene incluido en Node.js no necesitamos instalarlo con npm
import http from "http";

import fs from "fs";

// Creamos un servidor
// esta funcion se ejecuta cada vez que llega una peticion a un servidor
const server = http.createServer((req, res) => {
  // Leemos el archivo fichero.txt
  fs.readFile("fichero.txt", "utf-8", (error, datos) => {
    if (error) {
      res.writeHead(500, { "content-type": "text/plain; charset=utf-8" });
      res.end("Error al leer el archivo");
      return;
    }

    console.log("Contenido del archivo");
    console.log(datos);

    res.writeHead(200, { "content-type": "text/plain; charset=utf-8" });

    res.end(datos);
  });
});

// El servidor queda escuchando el puerto 8000

server.listen(8000, () => {
  // mensaje para confirmar en terminal que el servidor esta activo
  console.log("Servidor escuchando en http://localhost:8000");
});
```

Ejecutar:

node leerArchivo.js

Inicio de lectura

Fin del programa

Contenido del archivo:

Hola desde un archivo leído con Node.js

Este ejemplo nos muestra dos conceptos juntos: Node puede acceder al sistema de archivos y además muchas operaciones se trabajan de forma asíncrona

8. BLOQUE 7: ¿QUÉ PODEMOS HACER CON NODE?

Node no se limita a imprimir mensajes en consola. Su uso real aparece cuando construimos aplicaciones que reciben peticiones, entregan datos, se conectan a bases de datos y se integran con otros servicios.

Mapa de posibilidades

Node.js permite:

- Crear APIs REST
- Conectarse a bases de datos
- Consumir APIs externas
- Leer y escribir archivos
- Usar WebSockets
- Construir aplicaciones web completas
- Automatizar pruebas

Mini dinámica

Pedirles asociar cada necesidad con una capacidad de Node:

Guardar productos -> Base de datos / File System

Mostrar datos al frontend -> API REST

Leer configuración local -> File System

Chat en tiempo real -> Socket

Consumir datos del clima -> Consumir API externa

9. BLOQUE 8: INTRODUCCIÓN A EXPRESS

Con Node puro podemos crear servidores, pero Express nos ayuda a escribir menos código y organizar mejor las rutas. En proyectos reales esto es fundamental.

Comparación simple

Node.js:

Entorno que permite crear servidores.

Express:

Framework que simplifica rutas, middlewares, APIs y vistas.

10. BLOQUE 9: PRIMER SERVIDOR EXPRESS CON ES MODULES**Paso 1: instalar Express**

```
npm install express
```

Paso 2: crear index.js

```
import express from 'express';

const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Hola Mundo desde Express con ES Modules');
});

app.listen(PORT, () => {
  console.log(`Servidor escuchando en http://localhost:${PORT}`);
});
```

Paso 3: ejecutar

```
node index.js
```

Probar en navegador

```
http://localhost:3000
```

Explicación línea por línea

```
import express from 'express';
```

Importa Express usando ES Modules.

```
const app = express();
```

Crea una aplicación Express.

```
app.get('/', (req, res) => {
  res.send('Hola Mundo desde Express con ES Modules');
});
```

Define una ruta GET para /.

```
app.listen(PORT, () => {  
  console.log(`Servidor escuchando en http://localhost:${PORT}`);  
});
```

Levanta el servidor en el puerto indicado.